

Rapid #: -26773788

CROSS REF ID: **50653857740003606**

LENDER: **GZV (University of Wisconsin, Platteville) :: Main Library**

BORROWER: **CUD (University of Cambridge) :: Main Library**

TYPE: Book Chapter

BOOK TITLE: Communications and Networking: 12th International Conference, ChinaCom 2017, Xi'an, China, October 10-12, 2017, Proceedings, Part I

USER BOOK TITLE: Communications and Networking: 12th International Conference, ChinaCom 2017, Xi'an, China, October 10-12, 2017, Proceedings, Part I

CHAPTER TITLE: Parallel Symbol Timing Recovery Using FPGA for 600 Msps QPSK

BOOK AUTHOR: Bo Li, Lei Shu, Deze Zeng

EDITION:

VOLUME:

PUBLISHER: Springer Cham

YEAR: 2018

PAGES: 216-225

ISBN: 9783319781303

LCCN:

OCLC #:

Processed by RapidX: 5/12/2026 7:10:10 AM

This material may be protected by copyright law (Title 17 U.S. Code)



Parallel Symbol Timing Recovery Using FPGA for 600 Msps QPSK

Di Huang^{1(✉)}, Zhijie Wang², Jun Wang², and ZiYao Liu¹

¹ Beijing Institute of Technology, Beijing 100081, China
huangdibit@163.com, liuziyaochn@163.com

² Institute of Telecommunication Satellite, China Academy of Space
Technology, Xi'an 710000, China
1554264456@qq.com, bitwj@163.com

Abstract. This paper presents an efficient parallel symbol timing architecture for high data rate communications receivers. The presented architecture relies on a modified version of the classic Gardner loop, and it features a “multi-channel pipeline” interpolator that enables the symbol rate to be several times higher than the clock rate of the FPGA, hence maximize the achievable throughput. The presented timing recovery scheme is demonstrated on a Xilinx XC7VX690T FPGA at 150 MHz clock rate together with an ADC at 4.8 GHz sampling rate, for an QPSK data-stream at 600 Msps symbol rate. Also, it is observed the presented scheme occupies only 2% of the logic, storage and computational resources in the targeted FPGA. With minor modifications, our algorithm may be adapted for other Amplitude-Phase modulation constellations such as 8PSK, 16PSK or QAM.

Keywords: Symbol timing recovery · Interpolation · Multi-channel pipeline FPGA

1 Introduction

Symbol synchronization, a.k.a. timing recovery is one of the key techniques for digital communications receivers. In principle, symbol synchronization is to find the best sampling locations for each symbol [1], from an input baseband digital waveform at a sampling rate of f_{smp} , and usually f_{smp} is chosen such as $f_{\text{smp}} = N \cdot R_s$, where R_s is the symbol rate and N is a positive integral. Classic timing recovery methods in their original forms, for example the Gardner loop [2, 3], assumes at the receiver may perform Digital Signal Processing operations at a clock rate, say f_{clk} , equal to or larger than f_{smp} , and that is the very start-point for many practical digital receivers. However, increasing symbol rate means increasing information transmission bandwidth, which is highly demanded in numerous application contexts, ranging from global satellite communication systems to UAVs' 4K video transmission, just to name a few. As R_s rises to a level that f_{smp} or even R_s is larger than f_{clk} , symbol synchronization itself becomes a bottleneck to the overall design of the digital receiver, as now it must be implemented in a *parallel* way [4–7], and be capable of handling very high speed data throughput.

The main contribution of this paper is that we have designed an all-parallel symbol timing strategy for high symbol rate digital communications signals. Our strategy is based on an improved version of the classic Gardner loop, which comprises four principal components namely the Timing Error Detector (TED), the Loop Filter (LF), the Digital Controlled Oscillator (NCO) and the interpolator [8]. Among the above four, the parallel implementation of the interpolator poses the greatest challenge, and it is where we put our emphasis. In particular, we presented a “multi-channel pipeline” structure for the interpolator, where K identical channels all work at f_{clk} , and together they yield an overall throughput of $R_s = K \cdot f_{\text{clk}}$. On each channel, a Farrow interpolator is implemented by LUT, and the signal processing on the channel is designed as fully pipelined so as to achieve the maximum possible single-channel throughput f_{clk} . Low-complexity TED and LF are also discussed. The presented symbol timing recovery scheme is implemented on a Xilinx XC7VX690T FPGA working at $f_{\text{clk}} = 150 \text{ MHz}$. The FPGA accepts the output of a 4.8 GHz ADC, and performs symbol synchronization for a 600 Msps-QPSK signal at an Intermediate Frequency (IF) of $f_i = 1.2 \text{ GHz}$. Experimental results are provided to testify the effectiveness of the proposed parallel scheme. It is observed that the proposed scheme only takes 2% of the logic, computational and storage resources of the targeted FPGA.

The remainder of the paper is organized as follow. We first describe a scheme of the “parallel high data rate” demodulator and illustrate the module of timing recovery in which. We dedicate a section to analyses the “multi-channel pipeline” interpolator which enables the symbol rate to be four times higher than the clock rate of the FPGA. Next, we describe each module’ implementation details in the loop. The test result, such as constellation map and convergence curve are given followed by conclusion.

2 Parallel Demodulator Scheme

Figure 1 shows the main scheme of the parallel receiver. The Intermediate Frequency (IF) $f_i = 1.2 \text{ GHz}$, the 600 Msps IF signal is sampled at 4.8 GHz by EV10AQ190 One-channel Mode 10-bit A/D converter. The IF signal sampled by ADC is parallelized into 16 channels by a Xilinx FPGA built-in module, namely the Input Serialize & Reserialize (ISERDES), and then fed into 16 channel parallel FIFOs. All the subsequent operations are clocked at the a rate of $f_{\text{clk}} = 150 \text{ MHz}$.

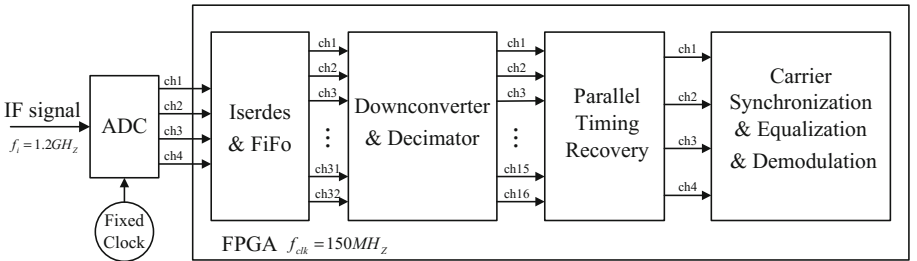


Fig. 1. The scheme of parallel high data rate receiver

Inside the FPGA, the 16-channel parallel Digital Downconverter (DDC) turns the IF sampled signal into digital I/Q baseband signal. A Decimator is inserted to reduce the sampling rate from 4.8 GHz to 2.4 GHz, so as to reduce the computational resource consumption. Even so, I/Q baseband signal at the rate of 2.4 GHz is still quite a big challenge for the subsequent timing recovery which has to work at a much lower clock rate of $f_{\text{clk}} = 150 \text{ MHz}$. And the solution is the emphasis of this paper, namely a high throughput timing recovery featuring a “multi-channel pipeline” architecture. The other signal processing modules that follow the timing recovery, namely carrier synchronization, equalization and demodulation, as can be seen in Fig. 1, are out of the scope of this paper, and hence shall not be discussed.

2.1 Parallel Timing Recovery Architecture

As a key part of the receiver, Parallel Timing Recovery turns 16 channels QPSK baseband signal into 4 channels at the FPGA clock rate $f_{\text{clk}} = 150 \text{ MHz}$. The block diagram of the symbol timing recovery is shown in Fig. 2.

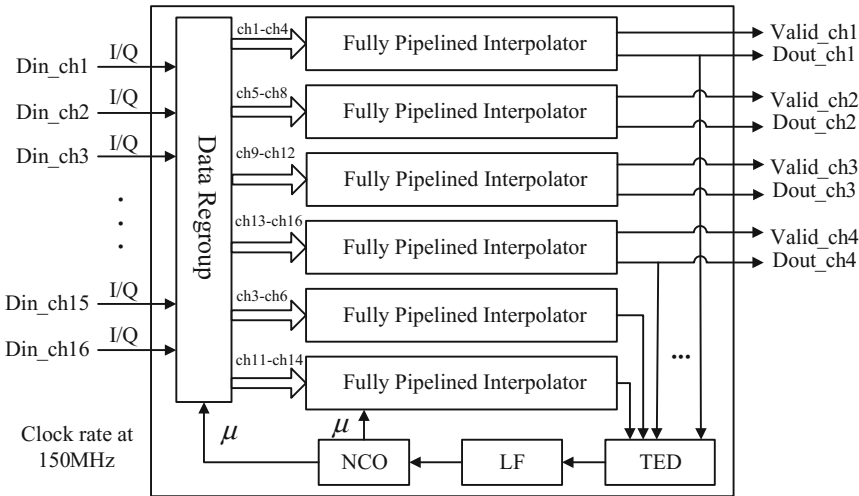


Fig. 2. Parallel timing recovery architecture

Data Regroup chooses 4 groups signal to Interpolators according to the fractional interval μ , and there are 4 channels in each group. Fully Pipelined Interpolator uses “multi-channel pipeline” structure to get 1 channel timing recovered signal from 4 channels signal, under the condition that the symbol rate R_s is 4 times higher than the clock rate f_{clk} . TED, LF and NCO are implemented with low complexity. Output data from interpolator also serve as the input of the TED. Loop Filter uses mean value of timing error estimation to compute control word, which afterwards would be sent to NCO. The fractional interval μ , which is generated from NCO to indicate the location, is sent to interpolator to correct the timing error.

2.2 Interpolator and Its Implementation

Gardner algorithm [9] is widely used in practical implementation for its simple structure. Polynomial interpolator can be efficiently implemented by Farrow Structure [10]. Farrow structure only need to transmit a fractional interval μ to compute an interpolation. This structure has been widely used in high speed digital system because it is fairly easy to implement. The calculation mathematical formulas of interpolation are as follows:

$$y(k) = \sum_{l=0}^N \mu_k^l v(l). \quad (1)$$

$$v(l) = \sum_{i=l_1}^{l_2} b_l(i)x(m_k i). \quad (2)$$

Where, $x(i)$ is the input of interpolator, and $b(i)$ is the coefficient of Farrow structure calculated by Lagrange formula, which is a fixed value which is not related to the fractional interval μ . This structure pre-computes the coefficients $b(i)$ and stores them in ROM [11, 12], which could be efficiently implemented by embedded Block RAM in FPGA. Figure 3 shows the “multi-channel pipeline” structure of interpolator.

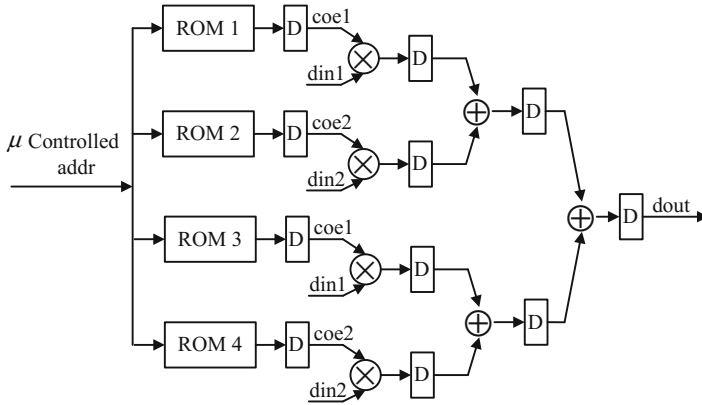


Fig. 3. Interpolator using ROM-based coefficients

Each channel has 4 sequential sub-modules. The first sub-module is implemented by LUT. Fractional interval μ would be used as the address to read the coefficients $b(i)$ from ROM. The second sub-module is used to obtain the product of input signal and coefficient. The third and fourth sub-modules are used to get summation as the output of interpolator.

2.3 Timing Error Detector and Its Implementation

This paper selects the Gardner timing error estimation algorithm [13] to estimate the timing error. Each symbol needs only two sampling points, also the timing error need not to be cumulated. The step is obtained by

$$err(k) = y(k - 1/2) * (y(k) - y(k - 1)). \quad (3)$$

Timing Error Detector adopts the parallel pipeline structure, in which clock rate is $f_{clk} = 150\text{MHz}$. $y(k)$ is the interpolated value. If there is no timing error, $y(k - 1/2)$ should be zero. Otherwise, the difference between $y(k)$ and $y(k - 1)$ is proportional to the value of error and also indicates the error direction. The final timing error is obtained by the mean value of two channels timing error.

2.4 Loop Filter and Its Implementation

The loop filter adopts a proportional integral structure which is always used in phase-locked loop. Figure 4 shows its implementation structure.

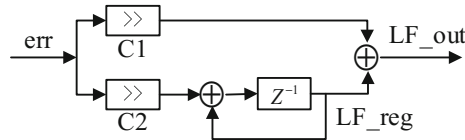


Fig. 4. The block diagram of loop filter

From Fig. 4, output of the loop filter Err_Lp is obtained by

$$LF_reg(k + 1) = LF_reg(k) + C2 * err(k). \quad (4)$$

$$LF_out(k + 1) = LF_reg(k + 1) + C1 * err(k). \quad (5)$$

$err(k)$ is the output of TED. In order to not use any multipliers, C1 and C2 are realized by shift operation. Through unceasingly debug in the practical simulation, C1 and C2 finally obtain the appropriate value.

2.5 Timing Controller and Its Implementation

Timing controller computes the enable signal and the fractional interval μ through internal addition algorithm. An incrementing NCO controls the μ . NCO which increases a certain step W every sampling time is a module 1 counter. Figure 5 shows the implementation structure of the timing controller. η is the content of the NCO register at one clock before the strobe signal, ε_0 could be an approximate representation of T_i/T_s with little performance loss. T_s is the sampling interval, T_i is the bit interval to the filter. Using the shift operation to replace multiplier is very convenient.

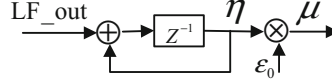


Fig. 5. Implementation structure of timing controller

2.6 Data Regroup and Its Implementation

According to Gardner algorithm, the data access location should move forward one more sampling data when $\mu > 1$, and it should move backward one sampling data when $\mu < 0$. Because of the parallel structure, the parallel input signal should be adjusted according to the fractional interval μ . This structure chooses 16 channels signal as interpolator inputs from 20 channels signal. Channel 1 to channel 4 are achieved from the previous sampling time and Channel 5 to channel 20 are achieved from this sampling time. Figures 6 and 7 show the implementation structure.

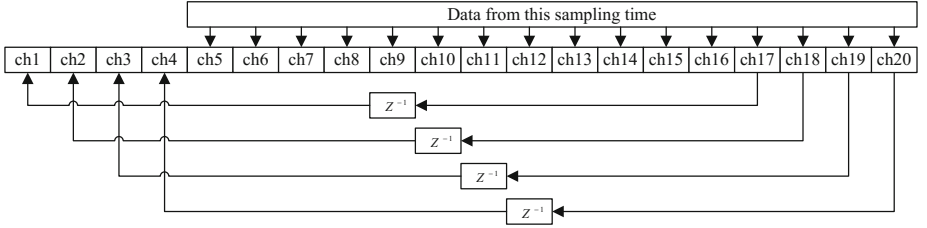


Fig. 6. Channel construction

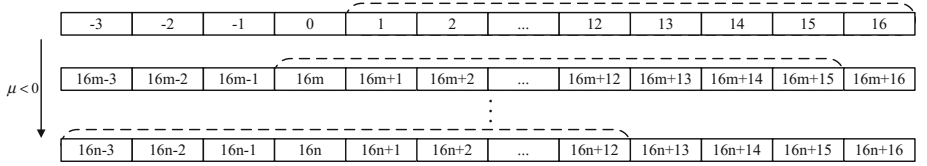


Fig. 7. Implementation structures of data regroup

The initial interpolation takes data(1) to data(16) as the interpolator inputs. Suppose $\mu < 0$ firstly occurred when interpolator has finished m th interpolation, data(16 m) to data(16 m + 15) should be inputted to interpolator. And so on, if the fourth $\mu < 0$ occurred after the n th interpolation, Data Regroup outputs data(16 n – 3) to data(16 n + 12) as the inputs of interpolator. If $\mu > 1$ occurred, channel 13 to channel 16 of interpolator's inputs are replaced by zero, and Dout_ch4 is not valid.

3 Simulation and Implementation Results

To verify the effectiveness of the symbol timing recovery scheme, we implement it on our own hardware platform, which is illustrated in Fig. 8. For a detailed list of the component in the used Software Defined Radio (SDR) hardware platform, the readers are referred to Table 1. As mentioned, the ADC samples the IF QPSK signal with a center frequency 1.2 GHz at a sampling rate of 4.8 GHz. The symbol rate of the IF signal is 600 Msps. Recalling Fig. 2, the timing recovery accepts a pair of down-converted IQ baseband signal at a sampling rate of 2.4 GHz (in the form of 16 channels, at the rate of 150 MHz each), and its outputs are 4 channel parallel optimally sampled symbol sequence at the rate of 150 MHz.

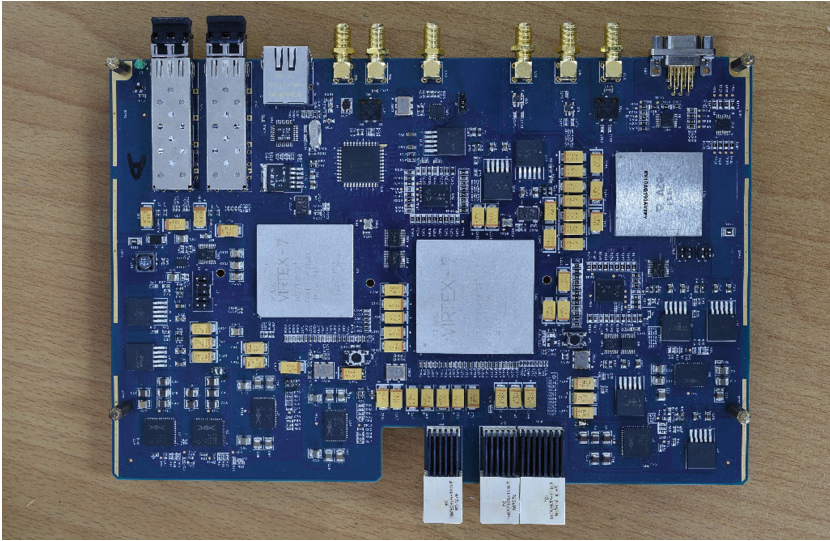


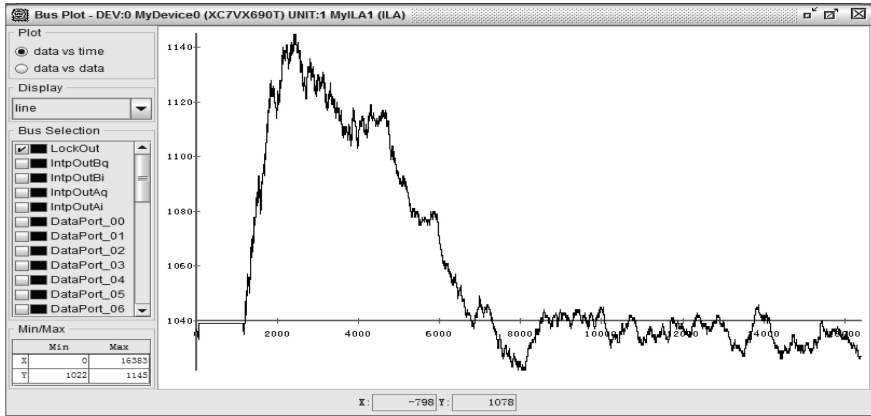
Fig. 8. Soft defined radio platform

Table 1 summarizes the resource utilization of the proposed symbol timing recovery scheme in the targeted FPGA model (Xilinx XC7VX690T FPGA). In particular, the overall resource consumption of the proposed timing recovery architecture is less than 2% of the total available amount in the targeted FPGA. The latest research [14] has been done for QPSK baseband signal with symbol rate of 45M. It's total implementation resources are 719 Slices and 18 embedded multipliers. By contrast, our proposed timing recovery schemes is not only suitable for high symbol rate but also very cost-efficient.

In Fig. 9 we give a typical convergence curve of the multi-channel pipeline timing recovery architecture, where the loop bandwidth is set as 0.001 of the symbol rate, i.e. 6 MHz, without loss of generality. The figure is produced by ChipScope, an integrated on-line diagnosis tool provided by Xilinx to monitor the real-time status of the FPGA's internal registers. In the figure the horizontal axis is the index of samples at the rate of

Table 1. Complexity comparison

Logic utilization	Xilinx XC7VX690T FPGA		
	Used	Available	Utilization
Number of slice	1860	75900	2%
Number of slice LUTS	3672	303600	1%
Number of block RAM/FIFO	4	2060	1%
Number of DSP48E1s	46	2800	1%

**Fig. 9.** Convergence curve of 600 Msps data rate

150 MHz, i.e., the FPGA's working clock. The vertical axis is the position of the optimal sampling epoch, which is discretized at a resolution of $T_{\text{smp}}/2^{10}$. As observed from the figure, it takes about 6000 samples (at the rate of 150 MHz) or 24000 symbols (at the rate of 600 Msps) for the loop to enter a convergent state, beginning from a cold start. Moreover, the stable vibration of the loop is about ± 20 according to the vertical axis in the figure, or $\pm 0.02 T_s$, translated by the relationship.

In Fig. 10 we plot the constellation with the output of the proposed timing recovery architecture. Figure 10(a) is generated by the raw data, i.e. the original output QPSK symbol samples after timing recovery. The constellation points are not very clear, like the input signal is noisy, but in fact we do not include noise in our test. The explanation for this phenomenon is when the symbol rate is as high as 600 Msps, even the coaxial cable between the test signal generator and the receiver will exhibit non-uniform amplitude & phase Response in the frequency domain, and thus cause Inter-Symbol-Interference (ISI). By comparison to Fig. 10(a), (b) is generated using the equalized version of the timing recovery outputs. The equalization in this example is carried out off-line by a Matlab routine, after the original raw data is collected from the FPGA by ChipScope. As can be seen in Fig. 10(b), the equalized constellation is very high-quality, indicating the proposed parallel timing recovery architecture works fine for 600 Msps QPSK digital communication signals.

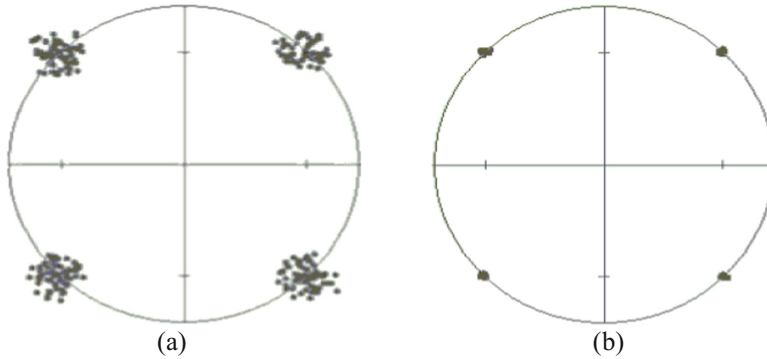


Fig. 10. Constellations of (a) output signal after timing recovery. (b) output signal after equalization.

4 Conclusion

In this paper we have designed a new all-digital “multi-channel pipeline” symbol timing recovery scheme, to bridge the gap between the extremely high symbol rate and the relatively low clock rate of the signal processors (usually FPGAs). The proposed scheme features a multichannel parallel pipelined architecture, and this allows it to be easily and efficiently implemented on FPGAs. As an example application of the proposed scheme, we have implemented it on a Software Defined Radio (SDR) platform for a 600 Msps QPSK IF signal. It is observed our timing recovery scheme could accept I/Q baseband signal at the rate of 2.4 GHz, based on a FPGA clocking rate of only 150 MHz. Also it is observed the proposed timing recovery algorithm only occupies about 2% of the logic, storage and computational resources in the targeted FPGA XC7VX690T. With some minor modifications, this scheme could be easily adapted to 8PSK, 16PSK or QAM and other higher data rate speed modem.

Acknowledgement. This work is supported by National Natural Science Foundation of China under contract No. 61471360

References

1. Ren, T., Petovello, M.: Collective bit synchronization for weak GNSS signals using multiple satellites. In: Position, Location and Navigation Symposium (2014)
2. Gardner, F.M.: Interpolation in digital modems. I. Fundamentals. *J. IEEE Trans. Commun.* **41**(3), 501–507 (1993)
3. Erup, L., Gardner, F.M., Harris, R.: Interpolation in digital modems. II. Implementation and performance. *J. IEEE Trans. Commun.* **41**(6), 998–1008 (1993)
4. Thompson, M., Jones, W.: An all digital 650-Mbps demonstration receiver for NASA’s high data rate satellite applications. In: Military Communications Conference (1992)

5. Schmidt, D., Lankl, B.: Parallel architecture of an all digital timing recovery scheme for high speed receivers. In: International Symposium on Communication Systems Networks and Digital Signal Processing (2010)
6. Lin, C., Shao, B., Zhang, J.: A high data rate parallel demodulator suited to FPGA implementation. In: International Symposium on Intelligent Signal Processing and Communication Systems (2011)
7. Lin, C., Zhang, J., Shao, B.: A high speed parallel timing recovery algorithm and its FPGA implementation. In: International Symposium on Intelligence Information Processing and Trusted Computing (2011)
8. Sheng, K., Anjum, M.R., Dida, M.A.: Principle of bit-synchronization loop. In: Multi-Topic Conference (2014)
9. Vesma, J., Saramaki, T.: Interpolation filters with arbitrary frequency response for all-digital receivers. In: International Symposium on Circuits and Systems (1996)
10. Farrow, C.W.: A continuously variable digital delay element. In: International Symposium on Circuits and Systems (1988)
11. Cardells-Tormo, F., Perez-Pascual, A., Torres-Carot, V.: Design of a DVB-S receiver in FPGA. In: Signal Processing Systems (2003)
12. Srinivasan, M., Chen, C.C., Grebowsky, G.: An all-digital high data-rate parallel receiver. *J. Surf. Sci.* **604**(15–16), 1294–1299 (1997)
13. Gardner, F.M.: A BPSK/QPSK timing-error detector for sampled receivers. *J. Trans. Commun.* **34**(5), 423–429 (1986)
14. Jian, Z., Nan, W., Jingming, K.: High speed all digital symbol timing recovery based on FPGA. In: International Conference on Wireless Communications, NETWORKING and Mobile Computing (2005)